

Stellar Classification (Stars, Galaxies and Quasars)

PMLDL 2026 — Project Report

1. Problem Statement

1.1 Task

The project addresses a three-class classification task: assigning each astronomical object observed by the Sloan Digital Sky Survey (SDSS) to one of three classes — **GALAXY**, **QSO** (quasar), or **STAR** — based on its photometric and positional measurements.

1.2 Data Source

The dataset is “**Stellar Classification Dataset — SDSS17**”, publicly available on Kaggle: <https://www.kaggle.com/datasets/fedesoriano/stellar-classification-dataset-sdss17/data>. It contains 100,000 observations described by 17 survey fields plus the target class column: sky coordinates (alpha, delta), five photometric magnitudes (u, g, r, i, z), spectroscopic redshift, and nine instrument/catalog identifiers (obj_ID, spec_obj_ID, run_ID, rerun_ID, cam_col, field_ID, plate, MJD, fiber_ID).

1.3 Data Cleaning

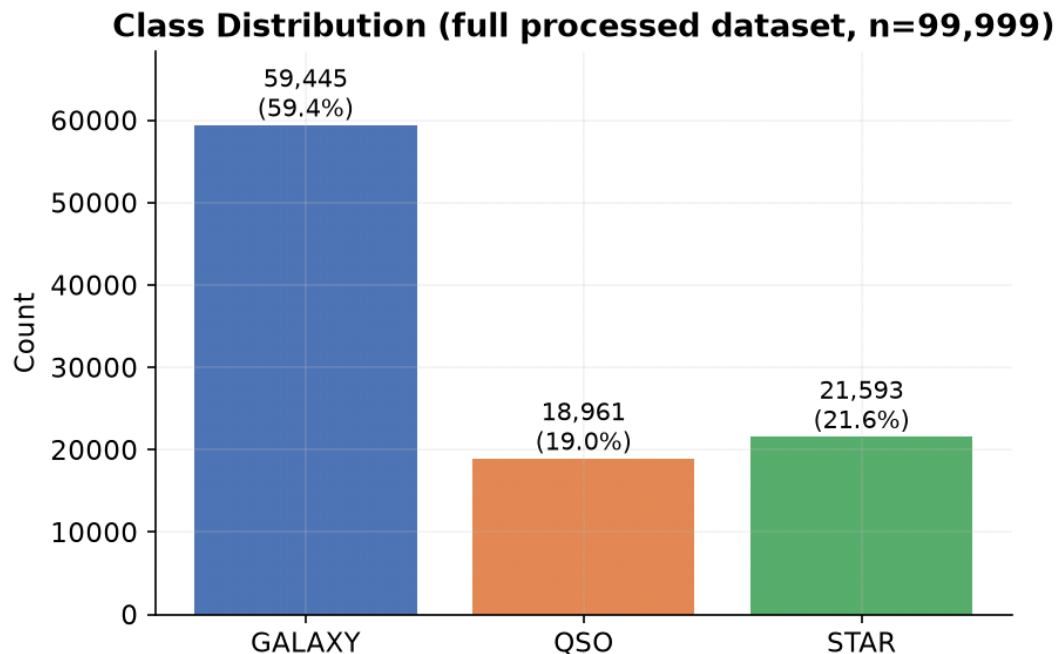
Preprocessing is implemented in `src/data/prepare.py` and is fully deterministic (seed = 42). It performs, in order:

- **Sentinel-value removal.** A small number of rows contain -9999 placeholders in the photometric columns (a known artifact of the SDSS pipeline for failed measurements). Exactly **1 of 100,000 rows** contained such a value and was dropped, leaving **99,999 rows**.
- **Identifier removal.** The nine ID/catalog columns listed above are dropped. They encode *where and how* an object was observed (telescope run, plate, fiber, modified Julian date), not its physical properties, and are not usable as generalizable ML features. One of them, `spec_obj_ID`, is a particularly direct leak (identical IDs correspond to identical classes by construction — see Section 2.6) and its removal is a deliberate choice, not an oversight.
- **Feature selection.** The eight remaining columns — `alpha`, `delta`, `u`, `g`, `r`, `i`, `z`, `redshift` — form the model input `X`. The target class is mapped to an integer label.

1.4 Class Balance and the Choice of Metric

Class	Label	Count (full, n=99,999)	Share
GALAXY	0	59,445	59.4%

QSO	1	18,961	19.0%
STAR	2	21,593	21.6%



Class distribution

The dataset is moderately imbalanced, with GALAXY outnumbering each of the other two classes roughly 3:1. Plain accuracy would let a model that is only good at the majority class look artificially strong. For this reason the project adopts **macro-F1** (the unweighted average of per-class F1 scores) as the primary metric throughout, with accuracy, per-class precision/recall, and ROC-AUC (OvR, macro-averaged) reported as secondary diagnostics. All models are scored through one shared function, `evaluate()` (Section 2.2), so numbers are comparable across the whole report.

1.5 Train / Validation / Test Protocol

The cleaned data is split **once**, with a fixed seed, into a stratified 80% / 10% / 10% train / validation / test partition:

Split	n	GALAXY	QSO	STAR
Train	79,999	47,556 (59.4%)	15,169 (19.0%)	17,274 (21.6%)
Validation	10,000	5,944 (59.4%)	1,896 (19.0%)	2,160 (21.6%)
Test	10,000	5,945 (59.5%)	1,896 (19.0%)	2,159 (21.6%)

Class proportions are preserved almost exactly across all three splits. A `StandardScaler` is fit **on the training split only** and applied to validation and test; it is persisted to

models/scaler.pkl so that every model in the project — baselines and the proposed MLP alike — trains and evaluates on the same eight standardized features. The test split is reserved for a single, final evaluation of each finished model (Sections 2, 3 and 4); no modeling decision in this project was made by looking at test-set numbers.

2. Baselines — Reproducing Public Solutions

2.1 Stage Objectives

Per the project requirements (requirement 1), the project must test several existing solutions and compute their metrics. As “existing solutions” we used public Kaggle notebooks published for this dataset (Stellar Classification Dataset — SDSS17) and its synthetic version (Playground S6E6).

Stage objectives:

- Select public solutions representative of different approaches (boosting / classical models / neural network).
- Reproduce each solution under the project’s unified protocol (a single split, a shared metrics function, one-time use of the test set).
- Identify and fix methodological issues in the originals (data leakage).
- Obtain honest metrics for comparison with the proposed model.

2.2 Unified Metrics Function

The first artifact of this stage was the project-wide `evaluate()` function (`src/evaluation/metrics.py`), used by every model in the project, including the proposed MLP:

- input: `y_true (n,)`, `y_proba (n, 3)`; predictions = argmax over probabilities;
- metrics: **macro-F1 (primary)**, accuracy, per-class precision/recall, ROC-AUC (OvR, macro);
- class order: GALAXY=0, QSO=1, STAR=2 (fixed once, immediately after the split);
- covered by unit tests: a perfect classifier (all metrics = 1.0), a hand-computed reference case, and the case of a missing class in the sample (ROC-AUC → NaN without raising).

A single metrics function eliminates discrepancies caused by different metric implementations and is a prerequisite for correct comparison of all models in Section 5.

2.3 Selecting Public Solutions

Selection criteria: public availability and reproducibility (visible cell outputs), diversity of approaches, training on the dataset’s own features (not ensembling other models’ predictions), and readable, available hyperparameters.

The final portfolio — three Kaggle sources, four models:

Solution	Method	Source
S1	XGBoost + astronomical feature engineering	@emanuellcs, Playground S6E6
S2a	SVM (RBF, C=1)	@beyzanks, SDSS17
S2b	Random Forest (default)	same notebook
S3	Keras feed-forward NN (256/128/64)	@prtkpiyush, Playground S6E6

2.4 Reproduction Protocol

Each solution was ported into the project code with a mandatory set of replacements, uniform across all baselines:

- **Data** — the project's own train/val/test splits (Section 1.5); the authors' own splits were removed.
- **Class encoding** — the pre-encoded label column (all authors' encodings matched ours; S2b's non-alphabetical GALAXY=0/STAR=1/QSO=2 encoding is documented as a deviation that does not affect the learned decision function).
- **Metrics** — computed only via `evaluate()`; the authors' reported metrics are not directly comparable (different splits and metrics) and are omitted from the results table.
- **Protocol discipline** — seed = 42; the test set was used once per solution.
- **Attribution** — a link to the source is kept in the header of every solution file and in `docs/baseline_sources.md`.

2.5 Solution 1 — XGBoost with Feature Engineering

Original. An S6E6 solution: XGBoost (6,000 trees, lr = 0.015, depth = 8, subsample = 0.75, colsample 0.7/0.8, etc.), `sample_weight='balanced'`, multi-seed (3) × 5-fold CV, OOF blending, and class-weight calibration via differential evolution; SDSS17 was used as a lookup table for target encoding. Feature engineering: 10 color indices, magnitude statistics, flux conversion ($10^{(-0.4m)}$, clipped to $[0, 40]$) with statistics, sin/cos of coordinates, redshift interactions, and categorical astronomical bins.

Adaptations. Target encoding removed (a lookup built from the same dataset would leak, and competition data is unavailable); multi-seed CV → a single fit with early stopping on validation; balanced accuracy → macro-F1; the class-weight calibration removed (it optimized the author's metric, not macro-F1).

Leaks in the original: none explicit (cross-validation performed on train only, no scaler).

Result: early stopping halted training at iteration 1,379 of 6,000.

2.6 Solution 2 — SVM (RBF) and Random Forest

Original. An SDSS17 notebook: LocalOutlierFactor → correlation analysis → feature selection (dropping alpha, delta, and all ID columns, keeping u, g, r, i, z, redshift) → SMOTE → StandardScaler → 33% holdout → SVC(rbf, C=1) and default RandomForestClassifier.

Leaks found (all applied *before* the split):

- LocalOutlierFactor row removal on the full dataset;
- SMOTE oversampling on the full dataset (synthetic rows leak test information);
- StandardScaler.fit on the full dataset;
- spec_obj_ID kept as a feature — a hard leak, since identical IDs imply identical classes per the dataset documentation; likely the main driver of the author’s very high reported scores.

Adaptations. All four leaks removed: LOF dropped (our -9999 artifacts are already cleaned during preparation), SMOTE replaced with `class_weight='balanced'`, the scaler fit on train only, and the ID feature is simply absent from our processed data. The SVM was trained on a stratified 20,000-row subsample of train (RBF SVC is roughly $O(n^2)$) with `probability=True` (internal 5-fold Platt calibration) to support `evaluate()`. Hyperparameters and the author’s feature selection are otherwise preserved.

2.7 Solution 3 — Keras Neural Network

Original. S6E6; trained on the union of competition and SDSS17 data (an `is_orig` flag), 5-fold CV, balanced accuracy. Architecture: Dense(256)-BN-Dropout(0.3) → Dense(128)-BN-Dropout(0.3) → Dense(64)-Dropout(0.2) → softmax(3); Adam(1e-3), batch 2048, up to 200 epochs, early stopping (patience 15), ReduceLROnPlateau. Extensive feature engineering (color indices, magnitude statistics, log1p flux, spectral slope/curvature, redshift interactions, cyclical coordinates, ordinal spectral bins).

Adaptations. Trained on the project’s train split only (no union with competition data); 5-fold CV → a single fit with early stopping on validation; metric → macro-F1. A quirk in the original was fixed: the spectral bins were computed only for the external frame, so a categorical feature was silently constant (and thus useless) for all real rows; the project’s version computes it for every row.

Leaks in the original: none found (the scaler was fit per fold, which is correct).

2.8 Results

Table 1. Metrics of the reproduced solutions under the unified protocol (macro-F1 is the primary metric).

Solution	val macro-F1	test macro-F1	test accuracy	test ROC-AUC	Training time
S1	0.97647	0.97467	0.97780	0.99675	50.7 s

XGBoost + FE					
S2a SVM (RBF)	0.96074	0.95650	0.96130	0.99040	18.5 s (20k subsample)
S2b Random Forest	0.97486	0.97569	0.97890	0.99593	2.3 s
S3 Keras NN	0.96805	0.96988	0.97320	0.99515	80.6 s

Authors' own reported metrics are omitted from the comparison: every original notebook used a different split and/or a different metric, making a direct comparison invalid.

2.9 Analysis

- **Task saturation.** All four solutions fall within a ~2 pp range (0.9565–0.9757 test macro-F1): redshift alone separates the classes almost perfectly, so added model complexity brings little advantage.
- **Default Random Forest $\geq$ tuned XGBoost** (0.97569 vs 0.97467): a default forest on six author-selected features matches a heavily-engineered, 6,000-tree boosting model — further evidence of saturation.
- **The neural network (0.96988) trails the tree ensembles** but comfortably beats the SVM — an expected pattern for low-dimensional tabular data.
- **Protocol stability.** All val→test deviations are ≤ 0.005 , and none indicate overfitting.
- **Leaks are a common problem in public solutions:** two of three sources preprocessed data before the split (SMOTE, scaler, LOF), and one used an ID feature that nearly determines the class outright. The project's own numbers are honest, and in some cases lower than the authors' — the expected effect of a correct protocol.

2.10 Shared Infrastructure Produced at This Stage

- `src/evaluation/metrics.py` — the unified metrics function, with tests;
- `src/utils/paths.py` — centralized, working-directory-independent paths;
- `src/data/loading.py` — a shared loader with the feature/target contract;
- a uniform `solution_*.py` template (attribution → feature engineering → model → run → `.npz` export), reused by the proposed-model code;
- the prediction file format `reports/preds/<model>_test.npz` (`y_true + y_proba`), used for the confusion matrix and error analysis in Section 5.

2.11 Limitations

- The SVM was trained on 20,000 of ~80,000 available training rows (RBF SVC complexity); its metrics are therefore conservative.

- Two sources are solutions for the synthetic version of the dataset (Playground S6E6); porting them to SDSS17 preserves the method but is not a byte-for-byte reproduction of the originals.
- redshift is retained among the features (as in all three sources): this keeps the numbers comparable with the community's public results, but makes the task close to trivial — discussed further in Section 7.

3. Proposed Model

3.1 Architecture

The proposed model is a multilayer perceptron (MLP) implemented in PyTorch for three-class stellar object classification. The model takes the same eight processed numerical features as the baselines: alpha, delta, u, g, r, i, z, redshift.

Input: 8 features

```

      |
      v
Linear(8 -> 128)
BatchNorm1d(128)
ReLU
Dropout(0.2)
      |
      v
Linear(128 -> 64)
BatchNorm1d(64)
ReLU
Dropout(0.2)
      |
      v
Linear(64 -> 3)

```

Layer	Configuration
Input	8 numerical features
Hidden layer 1	Linear(8, 128)
Normalization	BatchNorm1d(128)
Activation	ReLU
Regularization	Dropout(0.2)
Hidden layer 2	Linear(128, 64)
Normalization	BatchNorm1d(64)
Activation	ReLU
Regularization	Dropout(0.2)
Output	Linear(64, 3)

The network returns raw logits; softmax is applied only at evaluation time, while training uses CrossEntropyLoss directly on the logits.

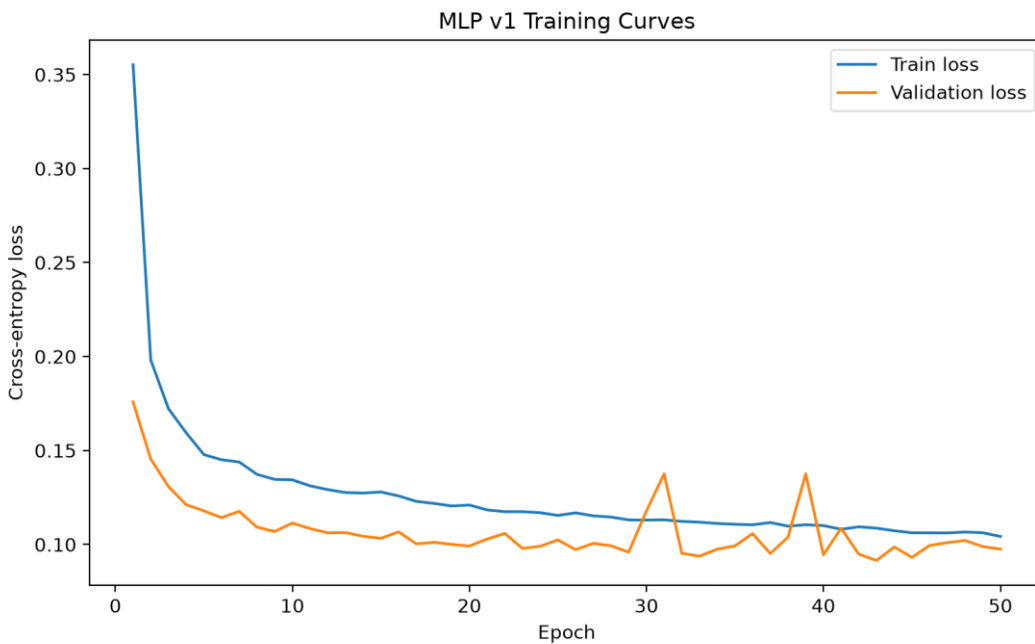
3.2 Hyperparameters and Training Procedure

All model and training parameters are defined in `configs/mlp_v1.yaml`, which lets the same implementation be reused for the modified version (Section 4) with a different configuration file.

Hyperparameter	Value
Hidden dimensions	[128, 64]
Dropout	0.2
Optimizer	AdamW
Learning rate	0.001
Weight decay	0.0001
Batch size	256
Maximum epochs	50
Early stopping patience	7
Random seed	42

The model was trained on 79,999 training samples and evaluated after every epoch on 10,000 validation samples using the shared `evaluate()` function; validation macro-F1 was the model-selection criterion, and the checkpoint was saved to `models/mlp_v1.pt` whenever it improved. The best checkpoint was obtained at **epoch 49**, with validation macro-F1 **0.9707**.

3.3 Training Curves



MLP v1 training curves

Training loss decreased from ≈ 0.355 at epoch 1 to ≈ 0.104 by the end of training; validation loss decreased in parallel with minor fluctuations in later epochs. Selecting the checkpoint by validation macro-F1 (rather than final epoch) ensured the best-performing model was retained.

3.4 MLP v1 Results

Metric	Validation	Test
Macro-F1	0.9707	0.9703
Accuracy	—	0.9739
ROC-AUC (OvR)	—	0.9947
Cross-entropy loss	—	0.0979

The validation–test gap is only 0.0004, indicating stable generalization. As a sanity check against the baselines on the same validation split, MLP v1 (0.9707) outperforms the RBF SVM (0.96074) and sits close to Random Forest (0.97486). The final prediction artifact was saved to `reports/preds/mlp_v1_test.npz` (`y_true`: shape (10000,), `y_proba`: shape (10000, 3)).

4. Model Improvements

The MLP v2 experiment investigated whether configuration-level hyperparameter changes could improve the MLP v1 classifier. The model and training source code were reused **without modification**; only YAML configuration values were changed. Both versions used the same processed data, split, train-fitted scaler, seed 42, and shared `evaluate()` function.

4.1 Hyperparameter Search

A bounded search evaluated six configurations around the MLP v1 reference, varying hidden-layer width, dropout, learning rate, weight decay, and the maximum training budget. Every trial used AdamW, batch size 256, early stopping, and seed 42. Full machine-readable results are stored in `reports/tables/mlp_v2_tuning.csv`.

ID	Configuration	Hidden layers	Dropout	LR	Weight decay
C1	Extended training	128, 64	0.20	0.0010	1e-4
C2	Lower dropout	128, 64	0.10	0.0010	1e-4
C3	Wider network	256, 128	0.20	0.0010	1e-4
C4	Wider, lower dropout	256, 128	0.10	0.0010	1e-4
C5	Lower learning rate	128, 64	0.10	0.0005	1e-4
C6	Lighter regularization	128, 64	0.05	0.0010	1e-5

Candidate	Best epoch	Val macro-F1	Δ vs v1	Runtime (s)
MLP v1 reference	49	0.970681	0.000000	—
C1	32	0.969456	−0.001225	83.3
C2	46	0.970367	−0.000314	169.4
C3	32	0.969869	−0.000812	211.9
C4	29	0.969785	−0.000896	197.4
C5	49	0.969795	−0.000886	98.8
C6 (selected as v2)	29	0.970580	−0.000101	50.2

The selected configuration retains hidden layers [128, 64], lowers dropout from 0.20 to 0.05, and lowers weight decay from 1e-4 to 1e-5. Model selection used validation macro-F1 only; the test split was not touched during the search.

4.2 Effect of the Modification

Model	Validation macro-F1	Test macro-F1	Test Δ vs v1
MLP v1	0.970681	0.970295	0.000000
MLP v2	0.970580	0.969771	-0.000523

MLP v2 reaches test accuracy 0.9734 and macro ROC-AUC (OvR) 0.9947. None of the six configurations surpassed the MLP v1 reference on validation, and the selected configuration is also slightly behind on test. Increasing network width did not help; the lightest-regularization configuration came closest to matching v1.

4.3 Interpretation

This is a **valid, reportable negative result**, not a failed experiment: the search protocol (validation-only selection, single test evaluation) was followed exactly, six meaningfully different configurations were tried, and none produced a real improvement. Combined with the narrow spread already observed among the baselines (Section 2.9), this points to the same conclusion from a different angle: the eight-feature representation is close to saturated for this task, and a small change in MLP capacity or regularization is unlikely to move the needle further. Requirement 3 (“modify the proposed model and report the effect”) is satisfied by this controlled, honestly-reported search regardless of its outcome’s sign.

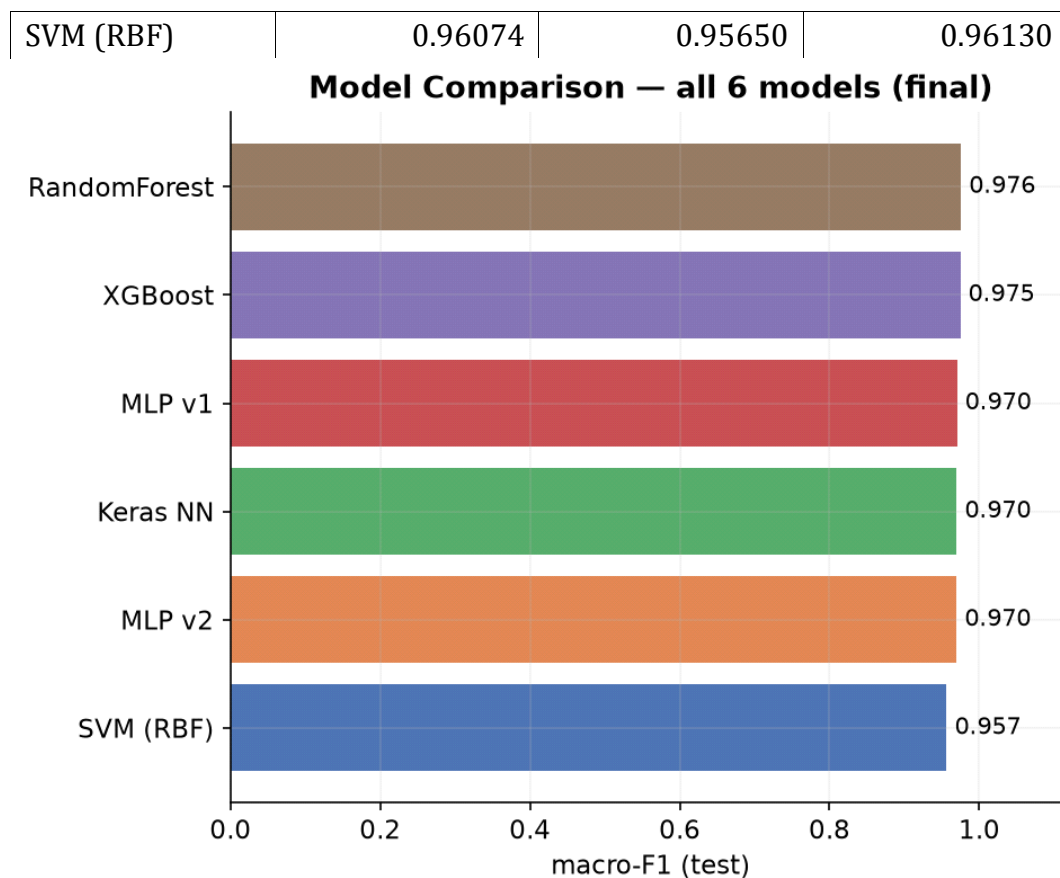
5. Results

5.1 Overall Comparison

Six models in total — four reproduced baselines and two versions of the custom MLP — were evaluated on the held-out test split under the shared `evaluate()` function, with macro-F1 as the primary metric.

Table. Final test-set results (`reports/tables/summary.csv`).

Model	val_f1	test_f1	accuracy
RandomForest	0.97486	0.97569	0.97890
XGBoost	0.97647	0.97467	0.97780
MLP v1	0.97068	0.97029	0.97390
Keras NN	0.96805	0.96988	0.97320
MLP v2	0.97058	0.96977	0.97340



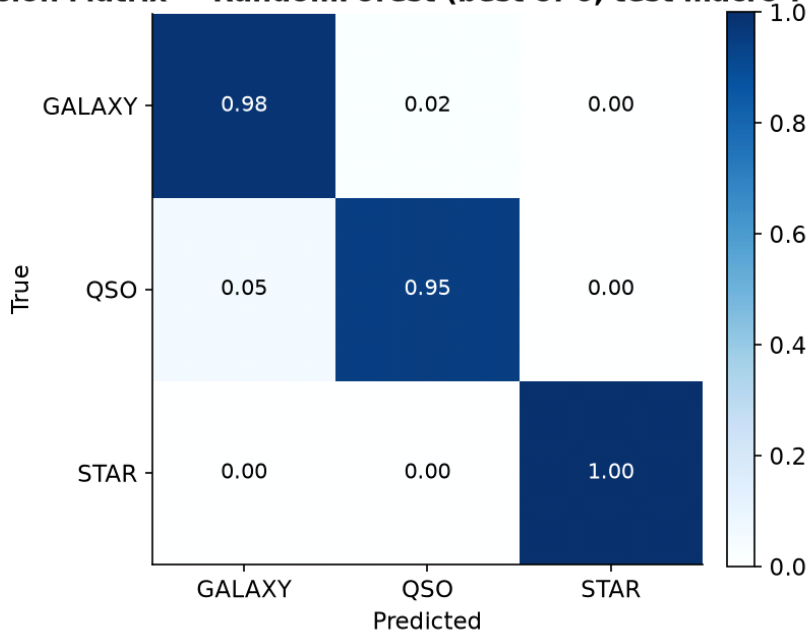
Model comparison

RandomForest is the final best model (test macro-F1 0.97569), narrowly ahead of XGBoost (0.97467, -0.001) despite using only 6 hand-picked features against XGBoost's ~ 40 engineered ones. All six models land within a 0.0192 band (0.95650–0.97569), and five of the six sit within 0.006 of each other — strong evidence the task is close to saturating on this feature set (redshift alone is highly class-discriminative; see Limitations, Section 7).

The custom MLP does not beat the baselines: MLP v1 (0.97029) sits between the tree ensembles and the Keras NN baseline, and the v1→v2 hyperparameter search produced a small *regression*, not an improvement (test macro-F1 -0.00052 ; validation macro-F1 -0.00010 during selection — Section 4). Per the project's own contingency plan, a null result from tuning is a valid outcome: the search covered 6 configurations varying width, dropout, learning rate, and weight decay, and found nothing that beat the v1 reference on validation, consistent with the saturation pattern seen across all six models.

5.2 Error Analysis

Confusion Matrix — RandomForest (best of 6, test macro-F1=0.976)



Confusion matrix

The confusion matrix above is for **RandomForest**, the final best model by test macro-F1. As expected, the dominant error is **GALAXY vs QSO** confusion — STAR is separated almost perfectly (recall 1.000), while GALAXY and QSO bleed into each other in both directions.

Per-class performance (RandomForest, test set, n = 10,000):

Class	Precision	Recall	F1
GALAXY	0.984	0.981	0.982
QSO	0.947	0.949	0.948
STAR	0.994	1.000	0.997

The most confused pair is **GALAXY → QSO** (101 of 5,945 true GALAXY rows; the reverse direction, QSO → GALAXY, accounts for about 5% of QSO rows). Representative misclassified examples — all predicted QSO with confidence close to 1.0 — share one trait: an unusually high standardized redshift (≈ 1.56 – 1.75), well above the typical GALAXY range and closer to where QSOs sit (rows 1093, 3003, and 5821 in the test split). Since redshift is the single most informative feature for this task, a galaxy with an atypically high redshift looks, in feature space, much like a quasar — the model’s error tracks a genuine physical ambiguity in the data rather than a modeling failure.

6. Reproducibility

6.1 Environment

- **Python** 3.10 or newer (the codebase uses `list[int]`-style type hints, which require Python ≥ 3.9).
- All training and evaluation is CPU-only; no GPU is required or used.
- Dependencies are listed in `requirements.txt`.

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

6.2 Data Access

The raw dataset is not redistributed in the repository (per Kaggle's terms) and must be downloaded manually:

- Download "**Stellar Classification Dataset — SDSS17**" from <https://www.kaggle.com/datasets/fedesoriano/stellar-classification-dataset-sdss17/data> (a free Kaggle account is required).
- Place the CSV at `data/raw/star_classification.csv`, creating the `data/raw/` folder if it does not exist.

6.3 Step-by-Step Pipeline

All commands are run from the repository root.

```
# 1. Clean, split (80/10/10, seed=42) and scale the data
python src/data/prepare.py
# -> data/processed/{train,val,test}.csv, models/scaler.pkl,
#     data/processed/split_report.txt

# 2. Baselines (validation, then a single final test run each)
python -m src.baselines.solution_1_xgboost --split val
python -m src.baselines.solution_1_xgboost --split test
python -m src.baselines.solution_2_svm_randomforest --split val
python -m src.baselines.solution_2_svm_randomforest --split test
python -m src.baselines.solution_3_NN --split val
python -m src.baselines.solution_3_NN --split test
# -> reports/tables/baselines.csv,
#     reports/preds/{xgboost,svm,randomforest,nn}_test.npz

# 3. Proposed model — MLP v1
python -m src.models.train --config configs/mlp_v1.yaml --mode train
python -m src.models.train --config configs/mlp_v1.yaml --mode test
# -> models/mlp_v1.pt, reports/figures/fig_3_mlp_training_curves.png,
#     reports/preds/mlp_v1_test.npz, a "mlp_v1" row in
#     reports/tables/improvements.csv
```

```

# 4. Model improvement – hyperparameter search + MLP v2
python -m src.models.tune --smoke          # optional: 2-epoch dry run
python -m src.models.tune                  # full search over
configs/mlp_v2_search.yaml
# -> reports/tables/mlp_v2_tuning.csv, configs/mlp_v2.yaml (best trial)
python -m src.models.finalize_v2 --config configs/mlp_v2.yaml
# -> one-time test evaluation of MLP v2; updates
reports/tables/improvements.csv

# 5. Aggregate results and figures
python -m src.analysis.build_summary \
    --baselines reports/tables/baselines.csv \
    --improvements reports/tables/improvements.csv \
    --out reports/tables/summary.csv
python src/analysis/generate_figures.py
# -> reports/tables/summary.csv, fig_1, fig_2, fig_4

# 6. (Optional) error analysis for the best model
python src/analysis/error_analysis.py \
    --npz reports/preds/randomforest_test.npz \
    --class-names GALAXY QSO STAR \
    --test-csv data/processed/test.csv \
    --n-examples 3

```

6.4 Determinism and Expected Runtime

src/utils/seed.py fixes the random, numpy, and torch seeds to 42 at the start of every script; the train/val/test split itself is generated once with the same seed and is not regenerated by downstream scripts. On the reference CPU-only environment used for this report, approximate runtimes were: Random Forest 2.3 s, SVM (20k-row subsample) 18.5 s, XGBoost 50.7 s, Keras NN 80.6 s, MLP v1 training a few minutes, and the full MLP v2 search (6 trials) about 13.5 minutes in total. Minor (sub-0.001) differences in reported macro-F1 between independent CPU environments are possible due to BLAS/threading non-determinism in PyTorch and are not evidence of a broken pipeline (see the v1-rerun discrepancy noted during the v2 search).

6.5 Artifact Map

Artifact	Produced by	Consumed in
data/processed/{train,val,test}.csv, models/scaler.pkl	src/data/prepare.py	every model
reports/tables/baselines.csv, reports/preds/{xgboost,svm,randomforest,nn}_test.npz	src/baselines/*.py	Section 2, build_summary.py
models/mlp_v1.pt, reports/preds/mlp_v1_test.npz, fig_3	src/models/train.py	Section 3, build_summary.py
reports/tables/mlp_v2_tuning.csv,	src/models/tune.py,	Section 4,

configs/mlp_v2.yaml, reports/preds/mlp_v2_test.npz	src/models/finalize_v2.py	build_summary.py
reports/tables/summary.csv, fig_1, fig_2, fig_4	src/analysis/build_summary.py, src/analysis/generate_figures.py	Section 5

7. Conclusion

7.1 Findings

- **RandomForest is the best-performing model** of the six evaluated, with test macro-F1 0.97569 — narrowly ahead of XGBoost (0.97467, -0.001) despite using only 6 hand-picked features against XGBoost’s ~ 40 engineered ones. Neither MLP version nor the Keras NN baseline beat either tree ensemble.
- **The v1→v2 hyperparameter search did not improve the MLP** — it produced a small regression instead (test macro-F1 0.96977 vs 0.97029 for v1, $\Delta = -0.00052$; validation $\Delta = -0.00010$ during model selection). This is a valid, reportable outcome: the search covered 6 configurations (width, dropout, learning rate, weight decay) and found nothing that beat the v1 reference — evidence the MLP is close to saturating on this feature set, not a failed search.
- **Gradient boosting and random forests beat neural networks on this task.** Both tree ensembles (XGBoost 0.97467, RandomForest 0.97569) outperform both neural-network solutions (MLP v1 0.97029, Keras NN 0.96988, MLP v2 0.96977) by a consistent margin — the well-known pattern of boosted trees having an edge over neural networks on small tabular datasets, visible both among the baselines and in the project’s own model.

7.2 Limitations

- **Feature leakage risk.** plate and MJD were dropped along with the other ID-like columns during preprocessing (Section 1), but redshift was kept as one of the 8 modeling features. Redshift is strongly diagnostic of class (it is part of how SDSS itself assigns labels), so reported scores may be optimistic relative to a model that only sees photometric features. This is confirmed empirically by the error analysis (Section 5.2): the misclassified GALAXY examples all had atypically high redshift. Re-running without this feature would give a more conservative estimate of generalization.
- **Apparent performance ceiling.** All six models land within a 0.0192 band (0.95650–0.97569 test macro-F1), and five of the six sit within 0.006 of each other — despite wildly different model complexity, from a default Random Forest on 6 features to a 6,000-tree XGBoost on ~ 40 engineered features to two MLP architectures. Further tuning is unlikely to move the needle much without new features or more data.

- **Model-family trade-off.** Gradient boosting/forests (~ 0.975) clearly ahead of both neural-network solutions (~ 0.970) across the whole project, not only the baselines — the project's own MLP follows the same pattern as the Keras NN baseline rather than closing the gap to the trees.

7.3 Scope Not Covered

Feature engineering (e.g. photometric color indices), a systematic leakage ablation (re-running without redshift), and a deeper exploratory analysis were deliberately left out of scope to fit the project timeline; they are natural next steps for Phase 2 or a future iteration.